



UNIT 2: CONTROL STATEMENTS

Prachi Surlakar
Assistant Professor
Computer Engineering Department
Goa College of Engineering

Unit - 2

Introduction: Features of Python, execution of Python programs, Python virtual machines, memory management, garbage collection, comparison between C and Python.

Data Types: Comments, docstrings, built-in data types, strings, sets, literals, user-defined data types, constants, identifiers, reserved words and naming conventions in python.

Operators: Arithmetic, assignment, unary, relational, logical, Boolean, bitwise, membership, identity operators, operator precedence and associativity.

Control statements: if, if-else, if-elif else, while, for, nested loops, break, continue, pass, assert and return statements

12

CONTROL STATEMENTS

Control statements are statements which can control or change the flow of execution.

Control statements available in python

- if
- if... else
- if...elif....else
- while loop
- for loop
- else suite
- break statement
- continue statement
- pass statement
- assert statement
- return statement

CONTROL STATEMENTS

Control statements

1. **if**
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The if statement

Syntax:

```
if condition:  
    statements
```

1. Python program to express a digit in a word.

```
# understanding if statement  
num=1  
if num==1:  
    print("One")
```

OUTPUT:

One

CONTROL STATEMENTS

Control statements

1. **if**
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The if statement

2. Program

```
str = 'Yes'
if str == 'Yes':
    print("Yes")
    print("This is what you said")
    print("Your response is good")
```

OUTPUT:

```
Yes
This is what you said
Your response is good
```

Note: While writing a group of statements we should write them all with proper indentation.

Indentation: spaces left before the statements.

PRACHI SURLAKAR

Indentation

- Indentation refers to the spaces that are used in the beginning of a statement.
- The statements with **same indentation** belong to the same group called a **suite**.
- By default python uses 4 spaces but it can be increased or decreased by the programmer.

```
if x==1:  
— print('a')  
— print('b')  
— if y==2:  
—— print('c')  
—— print('d')  
print('end')
```

6.1: The Effect of Indentation

Output when x=1, y=0	Output when x=1, y=2	Output when x=0, y=2
a b end	a b c d end	end PRACHI SURLAKAR

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The if ... else statement

Syntax:

```
if condition:  
    statements1  
else:  
    statements2
```

Program to check if a number is even or odd.

```
# to know if a given number is even or odd  
x=10  
if x % 2 == 0:  
    print(x, " is even number")  
else:  
    print(x, " is odd number")
```

OUTPUT:

```
10 is even number
```

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The if ... else statement

Program to check if a number is even or odd.(Accept number from keyboard.)

```
# to know if a given number is even or odd
x = int(input("Enter a number: "))
if x % 2 == 0:
    print(x, " is even number")
else:
    print(x, " is odd number")
```

OUTPUT:

```
Enter a number: 11
11 is odd number
```


CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The if ... else statement

Program to test whether a given number is from 1 to 10

```
# using 'and' in if ... else statement
x = int(input('Enter a number: '))
if x>=1 and x<=10:
    print("You typed", x, "which is between 1 and 10")
else:
    print("You typed", x, "which is below 1 or above 10")
```

OUTPUT:

```
Enter a number: 9
You typed 9 which is between 1 and 10
```

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. **if...elif....else**
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The if ...elif ... else statement

- Used when programmer has to test multiple conditions and execute statements depending on those conditions.

Syntax:

```
if condition1:  
    statements1  
elif condition2:  
    statements2  
elif condition3:  
    statements3  
else:  
    statements4
```

It is possible to write
if...elif statement without
else

Python program to check if a given number is zero, positive or negative.

```
# to know if a given number is zero or +ve or -ve  
num=-5  
if num==0:  
    print(num, "is zero")  
elif num>0:  
    print(num, "is positive")  
else:  
    print(num, "is negative")
```

OUTPUT:

-5 is negative

PRACHI SURLAKAR

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. **if...elif....else**
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The if ...elif ... else statement

Python program to accept a numeric digit from keyboard and display in words.

```
# to display a numeric digit in words
x = int(input('Enter a digit: '))
if x==0: print("ZERO")
elif x==1: print("ONE")
elif x==2: print("TWO")
elif x==3: print("THREE")
elif x==4: print("FOUR")
elif x==5: print("FIVE")
elif x==6: print("SIX")
elif x==7: print("SEVEN")
elif x==8: print("EIGHT")
elif x==9: print("NINE") # else part not compulsory
```

OUTPUT:

```
Enter a digit: 5
FIVE
```


CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. **while loop**
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The while loop

Executes a group of statements several times repeatedly.

If the condition is true it will execute the statements written after colon(:)

If the condition is false then it will come out of the while loop.

SYNTAX: `while condition:
 statements`

Write a python program to display numbers from 1 to 10

```
# to display numbers from 1 to 10
x=1
while x<=10:
    print(x)
    x+=1
print("End")
```

OUTPUT:

```
1
2
3
4
5
6
7
8
9
10
End
```

x	x<=10	print	Increment x
1	1<=10	1	x=1+1=2
2	2<=10	2	x=2+1=3
3	3<=10	3	x=3+1=4
...		...	
10	10<=10	10	x=10+1=11
11	11<=10 NO Terminate loop	End	

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. **while loop**
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The while loop

Determine the output of the following

```
x=1
while x<=10:
    print(x)
    x+=1
    print("End")
```

OUTPUT:

```
1
End
2
End
3
End
4
End
5
End
6
End
7
End
8
End
9
End
10
End
```


CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. **while loop**
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The while loop

Write a python program to display even numbers from 100 to 200

```
x=100
while x<=200:
    print(x)
    x+=2
```

OUTPUT:

```
100
102
104
:
:
200
```

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. **while loop**
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The while loop

Write a python program to display even numbers from m to n. Take m and n values from the user.

```
# to display even numbers between m and n
m, n = [int(i) for i in input("Enter minimum and maximum range: ").split(',')]
x=m # start from m onwards
if x % 2 !=0 : # if x is not even, start from next number
    x=x+1
```

```
while x>=m and x<=n:
    print(x)
    x+=2
```

OUTPUT:

```
Enter minimum and maximum range: 11,20
12
14
16
18
20
```

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
- 5. for loop**
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The for loop

SYNTAX:

```
for var in sequence:  
    statements
```

Program to display each character from a string

```
# to display each character from a string  
str='Hello'  
for ch in str:  
    print(ch)
```

OUTPUT:

```
H  
e  
l  
l  
o
```


CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
- 5. for loop**
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The for loop

SYNTAX:

```
for var in sequence:  
    statements
```

Program to display each character from a string

```
# to display each character from a string  
str='Hello'  
for ch in str:  
    print(ch)
```

OUTPUT:

```
H  
e  
l  
l  
o
```

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
- 5. for loop**
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The for loop

Write a program to display numbers from 1 to 10 using range() object.

```
# to display odd numbers between 1 and 10  
for i in range(1, 10, 2):  
    print(i)
```

OUTPUT:

```
1  
3  
5  
7  
9
```


CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
- 5. for loop**
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The for loop

Write a program to display numbers from 10 to 1 in descending order

```
# to display numbers in descending order
for x in range (10, 0, -1):
    print(x)
```

Write a program to display elements of a list using for loop.

```
# take a list of elements
list = [10,20.5,'A','America']

# display each element from the list
for element in list:
    print(element)
```


CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
- 5. for loop**
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The for loop

Write a program to display and find the sum of a list of numbers using a for loop.

```
# to find sum of list of numbers using for.  
# take a list of numbers  
list = [10,20,30,40,50]  
sum=0 #initially sum is zero  
for i in list:  
    print(i) #display the element from list  
    sum+=i #add each element to sum  
print('Sum= ', sum)
```

OUTPUT:

```
10  
20  
30  
40  
50  
Sum= 150
```

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The infinite loop loop

Example 1

```
i=1  
while i<=10:  
    print(i)
```

Example 2

```
while(True):  
    print("Hai")
```


NESTED LOOPS

- Write one loop inside another loop

Example:

- Outer for loop repeats for 3 times changing values from 0 to 2 and inner for loop repeats 4 times changing values from 0 to 3

```
for i in range(3): # i values are from 0 to 2
    for j in range(4): # j values are from 0 to 3
        print('i=', i, '\t', 'j=', j) # display i and j values
```

After first iteration

i=0 j value will change from 0 to 3

```
i= 0 j= 0
i= 0 j= 1
i= 0 j= 2
i= 0 j= 3
```

After second iteration

i=1, j value will change from 0 to 3

```
i= 1 j= 0
i= 1 j= 1
i= 1 j= 2
i= 1 j= 3
```

After third iteration

i=2, j value will change from 0 to 3

```
i= 2 j= 0
i= 2 j= 1
i= 2 j= 2
i= 2 j= 3
```


PATTERNS

```
*  
**  
***  
****  
*****
```

```
for i in range(1,6):  
    print('*' * (i))
```

i	print('*', * i)	
1	*	i=1
2	**	i=2
3	***	i=3
4	****	i=4
5	*****	i=5

Note: range(1,6) will create numbers as 1,2,3,4,5

PATTERNS

```
      *
     * *
    * * *
   * * * *
  * * * * *
```

i	print(' '*n,end=' ')	print('*'*(i))
1	4 spaces	*
2	3 spaces	* *
3	2 spaces	* * *
4	1 space	* * * *
5	0 space	* * * * *

n=4

```
for i in range(1,6):
    print(' '*n,end=' ')
    print('*'*(i))
    n=n-1
```


PATTERNS

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
for i in range(5):
    print('*' * 5)
```

range(5) will create numbers from 0 to 4

i	print('*' * 5)
0	* * * * *
1	* * * * *
2	* * * * *
3	* * * * *
4	* * * * *

PATTERNS

* * * * *

* * * *

* * *

* *

*

```
for i in range(5, 0, -1):  
    print('*' * (i))
```

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. **else suite**
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The else suite

In python we can use else statement with for loop or while loop

for with else

```
for(var in sequence) :  
    statements  
else:  
    statements
```

while with else

```
while(condition):  
    statements  
else:  
    statements
```

- Statements written after else are called as suite.
- Else suite will always be executed irrespective of the statements in the loop are executed or not.

```
for i in range(5):  
    print("Yes")  
else:  
    print("No")
```

OUTPUT:

Yes
Yes
Yes
Yes
Yes
No

```
for i in range(0):  
    print("Yes")  
else:  
    print("No")
```

OUTPUT:

No

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. **else suite**
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement

The else suite

Write a python program to search an element in a list of elements

```
# searching for an element in a list
group1 = [1,2,3,4,5]    #take a list of elements
search = int(input('Enter element to search: '))
for element in group1:
    if search == element:
        print('Element found in group1')
        break # come out of for loop
else:
    print('Element not found in group1') # this is else suite
```

OUTPUT 1:

```
Enter element to search: 4
Element found in group1
```

OUTPUT 2:

```
Enter element to search: 6
Element not found in group1
```

PRACHI SURLAKAR

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. **break statement**
8. continue statement
9. pass statement
10. assert statement
11. return statement

break statement

Break statement will come out of the loop

WITHOUT BREAK

```
x = 10
while x>=1:
    print ('x= ', x)
    x-=1
print("Out of loop")
```

OUTPUT:

```
x= 10
x= 9
x= 8
x= 7
x= 6
x= 5
x= 4
x= 3
x= 2
x= 1
Out of loop
```

x	x>=1	Print x	x-=1
10	10>=1	x=10	9
9	9>=1	x=9	8
8	8>=1	x=8	7
...	...	...	...
1	1>=1	x=1	0
0	Not satisfied		

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. **break statement**
8. continue statement
9. pass statement
10. assert statement
11. return statement

break statement

Break statement will come out of the loop

WITH BREAK

```
x = 10
while x>=1:
    print ('x= ', x)
    x-=1
    if x==5: # if x
        break
print("Out of loop")
```

OUTPUT:

```
x= 10
x= 9
x= 8
x= 7
x= 6
Out of loop
```

x	x>=1	Print x	x-=1	if(x==5)
10	10>=1	x=10	9	9==5
9	9>=1	x=9	8	8==5
8	8>=1	x=8	7	7==5
7	7>=1	x=7	6	6==5
6	6>=1	x=6	5	5==5 yes stop

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. **break statement**
8. continue statement
9. pass statement
10. assert statement
11. return statement

break statement

Determine the output of the following

```
x = 10
while x >= 1:
    if x == 5:
        x -= 1
        break
    print('x =', x)
    x -= 1
print("out of the loop")
```

OUTPUT:

```
x= 10
x= 9
x= 8
x= 7
x= 6
out of the loop
```

x	x>=1	if(x==5)	Print x	x-=1
10	10>=1	10==5	x=10	9
9	9>=1	9==5	x=9	8
8	8>=1	8==5	x=8	7
7	7>=1	7==5	x=7	6
6	6>=1	6==5	x=6	5
5	5>=1	5==5 x=4 break		

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. **continue statement**
9. pass statement
10. assert statement
11. return statement

continue statement

Continue with the loop without executing remaining statements

```
x = 10
while x >= 1:
    if x == 5:
        x -= 1
        continue
    print('x =', x)
    x -= 1
print("out of the loop")
```

OUTPUT

```
x = 10
x = 9
x = 8
x = 7
x = 6
x = 4
x = 3
x = 2
x = 1
out of the loop
```

x	x>=1	if(x==5)	Print x	x-=1
10	10>=1	10==5	x=10	9
9	9>=1	9==5	x=9	8
8	8>=1	8==5	x=8	7
7	7>=1	7==5	x=7	6
6	6>=1	6==5	x=6	5
5	5>=1	5==5 x=4 continue		
4	4>=1	4==5	x=4	3

```
for number in range(1, 6):  
    if number == 3:  
        continue  
    print(number)
```

1
2
4
5

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. **pass statement**
10. assert statement
11. return statement

pass statement

- Pass statement does not do anything.
- We use pass statement when we need a statement syntactically but we do not want to do any operation.

Write a python program to retrieve only negative numbers from a list

```
# Retrieving only negative numbers from a list
num = [1,2,3,-4,-5,-6,-7, 8, 9]
for i in num:
    if(i>0):
        pass # we are not interested
    else:
        print(i) # this is what we need
```

OUTPUT:

```
-4
-5
-6
-7
```

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
- 10. assert statement**
11. return statement

assert statement

- Assert statement is used to check if a particular condition is fulfilled or not.

SYNTAX:

assert expression, message

Example:

assert x>0, "Wrong input entered"

- If assert condition is **true** then the **next statement will be executed** else it will **display assertion error** along with the **message wrong input entered**.

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
- 10. assert statement**
11. return statement

assert statement

Program to assert that user enters number greater than zero

```
# understanding assert statement
x = int(input('Enter a number greater than 0: '))
assert x>0, "Wrong input entered"
print('U entered: ',x)
```

OUTPUT:

```
Enter a number greater than 0: -2
Traceback (most recent call last):
  File "Demo.py", line 3, in <module>
    assert x>0, "Wrong input entered"
AssertionError: Wrong input entered
```

AssertionError in the above output is known as an exception.
Exception is an error that occurs during runtime.

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
- 10. assert statement**
11. return statement

assert statement

Write a python program to handle AssertionError exception that is given by the assert statement.

```
# to handle AssertionError raised by assert
x = int(input('Enter a number greater than 0: '))
try:
    assert(x>0) # exception may occur here
    print('U entered: ',x)
except AssertionError:
    print("Wrong input entered") # this is executed in case of
    # exception
```

OUTPUT:

```
Enter a number greater than 0: -5
Wrong input entered
```

CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. **return statement**

return statement

Function:

- Represents a group of statements to perform a task.
- Function starts with a keyword def

Example:

```
def sum(a, b):  
    function body
```

Body of the statements contains logic to perform the task.

```
def sum(a, b):  
    print(a+b)
```

- A function is executed only when it is called.
- We can call function as:

```
sum(5, 10)
```


CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. **return statement**

return statement

Function:

Write a python program which has a function to find the sum of two numbers

```
# a function to find sum of two numbers
def sum(a, b):
    print('Sum= ', a+b)

sum(5, 10) # call sum() and pass 5 and 10
sum(1.5, 2.5) # call sum() and pass 1.5 and 2.5
```

OUTPUT:

```
Sum= 15
Sum= 4.0
```


CONTROL STATEMENTS

Control statements

1. if
2. if... else
3. if...elif....else
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. **return statement**

return statement

Function using return statement:

Python program to write a function that returns the result of sum of two numbers.

```
# a function to return sum of two numbers
def sum(a, b):
    return a+b # result is returned from here

res = sum(5, 10)
print('The result is ', res)
```

OUTPUT:

```
The result is 15
```

CONTROL STATEMENTS

Write a python program to display prime number series.

```
# program to print prime numbers upto a given number
# accept upto what number the user wants
max = int(input("Upto what number? "))

for num in range(2, max+1):    # generate from 2 onwards till max
    for i in range(2, num):    # i represents numbers from 2 to num-1
        if (num % i) == 0:    # if num is divisible by i
            break            # it is not prime, hence go back and check next num
    else:
        print(num) # otherwise it is prime and hence display
```

Example:

Upto what number? 30

Max=30

for num in range (2, 31) # so num will hold values from 2 to 30

for i in range(2, num): # so i will hold values from 2 till num-1

Example: num=5 # we are checking if 5 is prime or not

5%2==0 NO

5%3==0 NO

5%4==0 NO

Example:

4%2==0 YES

break

Print 5

Fundamental Algorithms:

Generation of Fibonacci series

- Every number is the sum of preceding two numbers
- First 6 terms of Fibonacci series are:
0 1 1 2 3 5

Initialise first two values

first =0, second=1

Formulas:

next =first + second


first=second


second=next

Example:

We want to generate first 6 terms

3 variables: first, second, next

first=0 second=1 next=0+1=1



first=1



second=1

next=1+1=2



first=1



second=2

next=1+2=3



first=2



second=3

next=
2+3=5

Write a program to generate Fibonacci series

```
n=int(input("how many fibonaccis?"))
first =0
second=1
i=3
if n==1:
    print(first)
elif n==2:
    print(first, '\n', second)
else:
    print(first, '\n', second, sep=' ')
    while i<=n:
        next=first+second
        print(next)
        first=second
        second=next
        i=i+1
```

Example:

n=6(number of terms), first=0,second=1, i=3
if(6==1) ✗ if no of terms to be printed is only one

Else

Iteration 1	Iteration 2	Iteration 3	Iteration 4	Iteration 5
first=0,second =1, i=3 Print 0, : 3<=6 next=0+1=1 Print 1 first=1 second=1 i=3+1=4	4<=6 next=1+1=2 Print 2 first=1 second=2 i=4+1=5	5<=6 next=1+2=3 Print 3 first=2 second=3 i=5+1=6	6<=6 next=3+2=5 Print 5 first=3 second=5 i=6+1=7	7<=6 ✗ STOP

Output:

0 1 1 2 3 5

THANK YOU😊

by PRACHI SURLAKAR

PRACHI SURLAKAR